

Data Structures for Engineers and Scientists Using Python



Rakesh Nayak and Nishu Gupta



CRC Press
Taylor & Francis Group

Introduction to Python

LEARNING OBJECTIVES

After studying this chapter, the reader will be able to:

- Identify variables and keywords
- Read from the keyboard and display on the screen
- Understand different data types and their usage
- Understand different types of operators, their usage and precedence
- Understand different flow control statements and their usage
- Understand functions, their types and application
- Apply all the above concepts to write programs

A programming language is a formal computer language that is used to send commands and data to a machine. Many languages were created with specific goals in mind, such as data processing and scientific calculations. The Python programming language is a very versatile language that can be used for both data processing and scientific calculations.

In this chapter we will discuss some important concepts of Python that will be helpful in learning data structures using Python.

I.1 VARIABLES, IDENTIFIERS AND KEYWORDS

A variable is a space in computer memory that may be used to store values. The type of variable is defined by the data it contains, and Python automatically allocates memory space to each variable. Because of this, Python is known as a *dynamic typing language*. In Python, variables do not need to be declared before they may be utilized.

The name given to a variable or other entity, such as functions or objects, is its identifier. The terms “variables” and “identifiers” are not synonymous. The name of a variable acts as an identifier, but it also contains other properties such as value, type and scope. Readers should note the following:



1. The first thing to keep in mind is that Python is case sensitive. As a result, identifier X and identifier x are not the same thing.
2. Identifiers only comprise of alphabets, digits and underscores.
3. Identifiers must not begin with a number.
4. Identifier names and Python keywords cannot be the same.
5. Identifiers can only be characters, numbers or underscores. For instance, we may name an identifier 'hello there' and assign it a value.
6. When an underscore is used as the first character, it is understood as a special identifier.

1.1.1 Assigning values to variables

To reserve memory space, Python variables do not require explicit declaration. When you assign a value to a variable, the declaration occurs automatically. When assigning values to variables, the equal sign (=) is used.

Example

`i=5` refers to a variable 'i' that stores an integer data.

`i=5.5` refers to a variable 'i' that stores floating-point data. The same variable 'i' can be used as integer as well as a floating point number at different parts of the Python program. Python allows to assign a single value to several variables simultaneously.

Example

`i = j = k = 0` refers to the variables 'i', 'j' and 'k' and all of them are assigned to 0 in a single statement.

`i, j, k = 4, 5.5, 'Hello'` refers to the variables where 'i' is assigned the value 4, 'j' is assigned the value 5.5 and 'k' is assigned the value 'Hello', and all the assignments are done in a single statement.

Keywords

Keywords are those reserved words that cannot be used as a variable because these words are used by Python and have specific meaning. Some of the keywords are listed in Table 1.1.

Table 1.1 Keywords

if	for	else	elif
as	except	is	return
assert	exec	in	try
not	and	or	pass
yield	or	continue	import

1.2 INPUT AND OUTPUT

To get the input from the keyboard, the `input()` function is called. The general format of the input function is

```
variable_name = input(["Prompt Message"])
```

The prompt message (optional) is something that is used to show a message to the user before he/she types anything. It is basically used for better understanding of the program. Output statements are the statements that are used to show the result on the screen. For that, we use the *print()* statement. A simple print statement looks like

```
print("object",[sep = " ", end = "\n", file= sys.stdout,flush = false])
```

All non-keyword arguments are converted to strings like *str()* and written to the stream, separated by comma '*sep*' and followed by '*end*'. Both *sep* and *end* must be strings; use of *sep* and *end* is optional. If no objects are given, *print()* will just print a blank line.

Example 1

```
name = input ("Enter your name :")
print("Your name is ", name)
```

Output

```
Enter your name : "Shiva"
Your name is Shiva
```

When we get an input from the keyboard by using *input()*, it takes strings. To make it numeric we need to do type casting.

Example 2

```
no1 = int(input("Enter Number 1 : "))
no2 = int(input("Enter Number 2 : "))
sum = no1 + no2
print("The sum is ", sum)
```

Output

```
Enter Number 1 : 33
Enter Number 2 : 55
The sum is 88
```

When the user's input is to be converted to the required type depending on the input, the *eval()* function attempts to evaluate a string in the same way that the interactive shell would evaluate it.

Example 3

```
no1 = eval(input("Enter number 1 : "))
no2 = eval(input("Enter number 2 : "))
sum = no1 + no2
print("The sum is ", sum)
```

Output 1

```
Enter number 1 : 2
Enter number 2 : 3
The sum is 5
```

Output 2

```
Enter number 1 : "Hello"
Enter number 2 : " How are you !"
The sum is Hello How are you !
```

There are many escape characters that are used to format a *print()* statement (Table 1.2).

Table 1.2 Escape characters

Escape character	Meaning
\n (New line)	Used to shift the cursor control to the new line.
\t (Horizontal tab)	Used to shift the cursor to a couple of spaces to the right in the same line.
\' (Apostrophe or single quotation mark)	Used to display the single-quotation mark.
\\" (Double quotation mark)	Used to display the double-quotation mark.
\\ (Backslash)	Used to display the backslash character.

Example 4

```
print("I am learning Python \nand I am enjoying it")
print("I am learning Python \tand I am enjoying it")
print("I\'m learning Python and I\'m enjoying it")
print("I am learning \"Python\" and I am enjoying it")
print("I am learning Python \\ I am enjoying it")
```

Output

```
I am learning Python
and I am enjoying it
I am learning Python      and I am enjoying it
I'm learning Python and I'm enjoying it
I am learning "Python" and I am enjoying it
I am learning Python \ I am enjoying it
```

There are two keyword parameters that may be used with the *print()* method. It is required to understand the following guidelines in order to utilize it. A keyword argument is made up of three parts: a *keyword* that identifies the argument (*end* or *sep*), an *equal sign* (=) and a *value applied to that argument*. Any keyword argument must come after the final positional argument (this is very important).

The *end* keyword argument determines the characters the *print()* function sends to the output once it reaches the end of its positional arguments. This behavior can be changed too with the help of the second keyword argument that is named *sep* (derived from separator). Note that the argument's value may be an empty string also.

Example 5

```
print("I am learning Python","and","I am enjoying it.", sep =  
      "...", end = "***")
```

Output

```
I am learning Python...and...I am enjoying it.***
```

1.3 INDENTATION

In Python, braces do not act as a distinguishing feature of a block of code while identifying class, function definitions or flow control. Instead, Python uses indentation of lines to indicate blocks of code. The lines are indented the same distance from the line containing the block of code. Python follows the indentation rule in its markup. The indentation of code blocks, rather than symbols, makes programs easier to read. Programs can be read more easily without symbols, and indentation indicates the type of statement. Code blocks can include single statements as well.

1.4 COMMENT STATEMENT

Whenever we want a line to be completely ignored by a compiler/interpreter, it is referred to as a *comment* statement. We use # (sharp or hash symbol) to comment a single statement or a part of a statement. Python ignores all remaining characters that appear after the hash symbol. Multiple lines may be commented using the triple quotation mark ("""" comment """).

Example 6

```
# This is a single line comment  
nol = eval(input("Enter Something : ")) # Anything after HASH  
    Symbol  
    """ This is  
    an example of  
    multiple line comment"""  
print("You have entered ",nol)
```

Output

```
Enter Something : "Shiva"  
You have entered Shiva
```

1.5 STANDARD DATA TYPES

A data type indicates the type of data that the identifier is capable of storing. Because there are several data types available as shown next, the programmer can choose which one is most appropriate for their application. In Python, each data type is a class.

- Numeric
- Boolean
- String
- List
- Tuple
- Set
- Dictionaries

1.5.1 Numeric data type

Objects of numeric data types are created when we assign a numeric value to an identifier. Numerical values are stored in numeric data types. Python supports four different numerical data types:

- int (signed integers)
- long (long integers), they can also be represented in octal and hexadecimal.
- float (floating point real values)
- complex (complex numbers)

Example 7

[illegible]

Output

```
int data is 5
float data is 5.5
long integer data is 55555555555555555555
complex data is (4+5j)
```

1.5.2 Boolean data type

A binary variable that can have one of the two possible values – 0 (False or F) or 1 (True or T) – is called Boolean.

Example 8

```
a = True
b = False
a1 = bool(0)
b1 = bool(1)
print(a,a1)
print(b,b1)
```

Output

```
True False
False True
```

1.5.3 String data type

Strings are nothing more than a continuous combination of characters, numbers and symbols. A string is immutable; once a string is defined it cannot be changed. A pair of single quotes or a pair of double quotes can be used to assign a string in Python. The plus sign (+) is the string concatenation operator and the asterisk (*) is the repetition operator.

Example 9

```
str1 = "I am Learning Python"
str2 = 'I am enjoying it.'
no=2
print(str1)
print(str2)
print(str1 + str2)
print(str2 * no)
```

Output

```
I am Learning Python
I am enjoying it.
I am Learning PythonI am enjoying it.
I am enjoying it.I am enjoying it.
```

Substring of strings may be obtained by using the slice operator ([] and [:]), with indices starting at 0 while moving from left to right and starting at -1 while moving from right to left.

The general syntax is *string_Name=string[start : end : step]*, where the start is the starting index, the end is the ending index and step (optional) is the integer to skip before getting the next character. The start index must always be smaller than the end index, or else the function will return an empty string. Furthermore, the element in the end index is not included.

Example 10

```
str = "I am Learning Data Structure using Python"
print("The first character : ",str[0])
print("Sub string between index 0 and 13 : ",str[0:14])
print("Reverse Order : ",str[-1:-14:-1])
print("Every alternative character starting at index 0 : 
",str[0::2])
```

Output

```
The first character : I
Sub string between index 0 and 13 : I am Learning
Reverse Order : nohtyP gnisu
Every alternative character starting at index 0 : Ia erigDt
tutr sn yhn
```

There are several built-in methods available with string. Few frequently used methods are listed in Table 1.3.

Table 1.3 String methods

Method	Description
isalnum()	Returns true if the string has at least 1 character and all characters are alphanumeric and false otherwise.
isalpha()	Returns true if the string has at least 1 character and all characters are alphabetic and false otherwise.
isdigit()	Returns true if string contains only digits and otherwise false.
islower()	Returns true if the string has at least 1 cased character and all cased characters are in lowercase and false otherwise.
isnumeric()	Returns true if a unicode string contains only numeric characters and false otherwise.
isupper()	Returns true if the string has at least one cased character and all cased characters are in uppercase and false otherwise.
lower()	Converts all uppercase letters in string to lowercase.
max(str)	Returns the max alphabetical character from the string str.
min(str)	Returns the min alphabetical character from the string str.
replace(old, new [, max])	Replaces all occurrences of old in string with new or at most max occurrences if max given.
swapcase()	Inverts case for all letters in string.
upper()	Converts lowercase letters in string to uppercase.
zfill (width)	Returns original string left padded with zeros to a total of width characters; intended for numbers, zfill() retains any sign given (less one zero).
len(string)	Returns the length of the string.
find(str, beg=0, end=len(string))	Determines if str occurs in string or in a substring of string if starting index beg and ending index end are given returns index if found and -1 otherwise.

1.5.4 List

A list is a mutable sequence of objects. It means we can change (update, insert, delete) the objects in a list. It can have data of all different data types under one name.

The general syntax for a list is

List_Name = [object 1, object 2 , object 3 , ... , object n]

Just like elements of a string, the elements of a list can be accessed with an index. A sub-list can be obtained from a list by slicing.

Example 11

```

lst=[]
print("Empty List : ",lst)
str1 = "Python"
lst = list(str1)
print("List from a string : ",lst)
lst = [4,4.5,"Hello",[1,2,3],(9,7,6),{'a','b','c'},{1:'aa',2:'bb'}]
print("Type of list : ",type(lst))
print("Data in list : ",lst)
print("Third element in the list ",lst[3])
print("1st to 4th element in the list : ",lst[1:5])

```

Output

```

Empty List : []
List from a string : ['P', 'y', 't', 'h', 'o', 'n']
Type of list : <class 'list'>
Data in list : [4, 4.5, 'Hello', [1, 2, 3], (9, 7, 6), {'a', 'c', 'b'}, {1: 'aa', 2: 'bb'}]
Third element in the list [1, 2, 3]
1st to 4th element in the list : [4.5, 'Hello', [1, 2, 3], (9, 7, 6)]

```

A new list can also be created from more than one existing list by the “+” (concatenation) operation and “*” (repetition) operation.

Example 12

```

lst1 = [1,2,3]
lst2 = [7,8,9]
print("New list with concatenation is ", lst1 + lst2)
print("New list with repetition is ", lst1 * 2)

```

Output

```

New list with concatenation is [1, 2, 3, 7, 8, 9]
New list with repetition is [1, 2, 3, 1, 2, 3]

```

There are several built-in methods available with lists. Few frequently used methods are listed in Table 1.4.

1.5.5 Tuple

A tuple is an immutable sequence of objects. It can have data of all different data types under one name.

The general syntax for a list is

Tuple_Name = (object 1, object 2, object 3, ..., object n)

Or

tuple_name = object 1, object 2, object 3, ..., object n

Table 1.4 List methods

Method	Description
<code>len()</code>	This method returns the length of a list.
<code>del(list_name[[start[:end]]])</code>	If there is no parameter passed, it deletes the entire list.
<code>max(List_name1,[List_name2])</code>	This function returns the maximum in a list
<code>min(List_name1,[List_name2])</code>	This function returns the minimum in a list.
<code>list.append(item)</code>	This method adds an item to the end of the list.
<code>insert(i, x)</code>	This method inserts an item at a given position.
<code>list.pop([i])</code>	This method removes the item which is at a given location and returns it.
<code>list.remove(x)</code>	This method searches the item in the list and removes it if it is found.
<code>list.extend(L)</code>	This function extends a list by appending all its items into a given list.
<code>list.index(x)</code>	This function returns the location of the first occurrence of the searched item in the list.
<code>list.count(x)</code>	This function returns the number of times the item appears in the list.
<code>list.sort reverse=True False, key=key)</code>	This function sorts the items of the list.
<code>list.reverse()</code>	This function reverses the elements of the list in place.
<code>list.copy()</code>	This function returns a copy of the given list.
<code>list.clear()</code>	This function removes all items from the list.

Just like elements of a string, the elements of a tuple can be accessed with an index. A sub-tuple can be obtained from a list by slicing.

To create a tuple with a single item, a comma is used after the element. All the methods that are available with a list are the same in a tuple except for `insert()` and deleting a few elements from a tuple.

Example 13

```
tup = ()
print("Empty Tuple : ",tup)
tup = (1,)
print("Tuple with one element: ",tup)
tup = (4,4.5,"Hello",[1,2,3] ,(9,7,6) ,{'a','b','c'} ,{1:'aa',2:'bb'})
print("Type of Tuple : ",type(tup))
tup1 = tup[1:6:2]
print("1st element to 6th alternative element of tuple: ",tup1)
tup1 = (1,2,3)
tup2 = (7,8,9)
print("The tuple-2 is ",tup2)
tup3 = tup1 + tup2
print("New tuple with concatenation is ",tup3)
tup3 = tup1 * 3
print("New tuple with repetition is ",tup3)
```

Output

```

Empty Tuple : ()
Tuple with one element: (1,)
Type of Tuple : <class 'tuple'>
1st element to 6th alternative element of tuple: (4.5, [1, 2, 3], {'a', 'c', 'b'})
The tuple-2 is (7, 8, 9)
New tuple with concatenation is (1, 2, 3, 7, 8, 9)
New tuple with repetition is (1, 2, 3, 1, 2, 3, 1, 2, 3)

```

1.5.6 Set

A set is an unordered and unindexed collection of unique elements. A set in Python is defined as a collection of elements in curly brackets.

Set_Variable_Name = { item1, item2, item3,...,itemn}

Set_Variable_Name = set([item1, item2, item3,...,itemn])

Example 14

```

s = {1}
s1 = set()
print("Empty set : ",s,s1)
print("Type is : ",type(s),type(s1))
lst= [1,2,3,4,5]
set_A=set(lst)
print("The elements are ", set_A, " the type is ", type(set_A))

```

Output

```

Empty set : {1} set()
Type is : <class 'set'> <class 'set'>
The elements are {1, 2, 3, 4, 5} the type is <class 'set'>

```

There are several built-in set operations available. Few frequently used methods are listed in Table 1.5.

Example 15

```

set_A = {1,2,3}
set_B = {3,4,5}
set_C = set_A | set_B
set_D = set_A & set_B
set_E = set_A ^ set_B
set_F = set_A.symmetric_difference(set_B)
print("Set A :",set_A)
print("Set B :",set_B)
print("Union of Set A and Set B :",set_C)
print("Intersection of Set A and Set B :",set_D)
print("Exclusive-or of Set A and Set B :",set_E)
print("Symmetric difference of Set A and Set B :",set_F)

```

Output

```
Set A : {1, 2, 3}
Set B : {3, 4, 5}
Union of Set A and Set B : {1, 2, 3, 4, 5}
Intersection of Set A and Set B : {3}
Exclusive-or of Set A and Set B : {1, 2, 4, 5}
Symmetric difference of Set A and Set B : {1, 2, 4, 5}
```

Table 1.5 Set operations

Method	Description
<code>set1.union(set2)</code> or <code>set1 set2</code>	This method returns a set of all those elements that are in either of the sets.
<code>Set1.intersection(Set2)</code> or <code>Set1 & Set2</code>	This method returns all those elements that are in both sets.
<code>set1.difference(Set2)</code> or <code>Set1 - Set2</code>	This method returns all those elements that are in the first set but not in the second set.
<code>Set1 ^ set2</code>	This operation returns all those elements that are either in one set or the other set but not in both.
<code>set1.symmetric_difference(Set2)</code>	This method returns all those elements that are in either of the sets, <code>set1</code> or <code>set2</code> , but not in both.
<code>Set_name.add(element)</code>	This method adds an element to the existing set.
<code>Set_name.discard(element)</code>	This method deletes an element from the set.
<code>Set1.isdisjoint(Set2)</code>	This method checks if any common elements are between two sets.
<code>Set1.issubset(Set2)</code>	This method checks if any one set is a subset of other sets.
<code>Set1.issuperset(Set2).</code>	This method checks if any one set is a superset of other sets.

1.5.7 Dictionary

A dictionary in Python is an unordered collection of items. A key is assigned to each item in a dictionary, and each key is connected with a value. The items in a dictionary are not arranged in any particular sequence. Instead of being regarded as a series of things, the dictionary is handled as a bag of stuff.

The general syntax is

Dictionary_Name = {*key_1:value_1, key_2:value_2,...,key_n: value_n*}

A dictionary starts and ends with a pair of curly brackets { }. The elements are filled within the curly brackets in the form of a key and a value pair separated by a colon. Each key-value pair is separated by comma. The elements inside a directory may have different types. The keys and the values in a directory may be numeric or string. Each key must be unique. It means

it is not possible to have more than one key of the same value. A dictionary is a one-way tool. It means, if a key is given, the corresponding value can be retrieved, but not vice versa. Refer to an item in a dictionary by its key.

Example 16

```
dict_A={}
print("Empty Dictionary with { }",dict_A," type is ",type(dict_A))
dict_B= dict()
print("The Empty dictionary with dict() ", dict_B, " and ", dict_B)
dict_C = {1:'RAMA', 2:'HARI', 3:'SHYAM', 4:'KRISHNA'}
print('The dictionary with { }', dict_C)
dict_D = dict([ [1,'DURGA'],[2,'SITA'],
[3,'RADHA'],[4,'SARASWATI']])
print("The dictionary is with dict() ", dict_D)
keys = [1,2,3,4]
values = ['RAMA', 'LAKSHMAN', 'BHARAT', 'SATRUGHNA']
dict_E = dict(zip(keys,values))
print("The dictionary with zip ", dict_E)
```

Output

```
Empty Dictionary with { } {} type is <class 'dict'>
The Empty dictionary with dict() {} and {}
The dictionary with { } {1: 'RAMA', 2: 'HARI', 3: 'SHYAM', 4:
'KRISHNA'}
The dictionary is with dict() {1: 'DURGA', 2: 'SITA', 3:
'RADHA', 4: 'SARASWATI'}
The dictionary with zip {1: 'RAMA', 2: 'LAKSHMAN', 3: 'BHARAT',
4: 'SATRUGHNA'}
```

There are various ways we can access the dictionary. Some of the ways are listed in Table 1.6.

Table 1.6 Dictionary methods to access key

<i>Method</i>	<i>Description</i>
<code>dictionary_Name[key]</code>	If the key of the dictionary is known, we can find its value.
<code>dictionary_Name.keys()</code>	This method returns a list built of all the keys within the dictionary.
<code>dictionary_Name.items()</code>	The method returns a list of tuples, where each tuple is a key–value pair.
<code>dictionary_name.get(key, [default=None])</code>	This method returns the value that is associated with the key.

Example 17

```
dict_A = {1:'RAMA', 2:'HARI', 3:'SHYAM', 4:'KRISHNA'}
print("The value with key = 2 ",dict_A[2])
print("The value with key=1 is ",dict_A.get(1))
print("The keys of the dictionary are ",dict_A.keys())
print("The items in the dictionary are ",dict_A.items())
```

Output

```
The value with key = 2 HARI
The value with key=1 is RAMA
The keys of the dictionary are dict_keys([1, 2, 3, 4])
The items in the dictionary are dict_items([(1, 'RAMA'), (2,
'HARI'), (3, 'SHYAM'), (4, 'KRISHNA')])
```

There are several built-in set operations available. Few frequently used methods are listed in Table 1.7.

Table 1.7 Dictionary methods

Method	Description
dictionary_Name.update(Another_Dictionary_Name)	This method updates a dictionary by using another dictionary.
sorted(dictionary_name).	This method returns a sorted key–value pair.
str(dictionary_Name).	This method returns a string of a given dictionary.
setdefault(key,[default_value=None])	This method is used to set a value of a key.

1.6 OPERATORS

An operator is a symbol in programming that generally symbolizes an action or process. Mathematical and logical symbols were used to create these symbols. An operator is a program that can manipulate a value or operand.

1.6.1 Arithmetic operators

Python provides all basic arithmetic operators. The symbols that are used for mathematical operations are +, -, *, /, **, %, //, and a unary minus sign (Table 1.8).

Table 1.8 Arithmetic operators

Operator	Meaning
+	Addition
-	Subtraction and unary minus
*	Multiplication
/	Division
**	Exponential
%	Modulo division
//	Integer division

Example 18

```

a = 10
b = 2
print("The value of a and b is ",a,b)
c = a + b
print(a," + ", b ," = ",c)
c = a - b
print(a," - ", b," = " ,c)
c = a * b
print(a," * ", b," = " ,c)
c = a / b
print(a," / ", b," = " ,c)
c = a ** b
print(a," ** ",b," = " ,c)
c = a // b
print(a," // ",b," = " ,c)

```

Output

```

The value of a and b is 10 2
10 + 2 = 12
10 - 2 = 8
10 * 2 = 20
10 / 2 = 5.0
10 ** 2 = 100
10 // 2 = 5

```

1.6.2 Assignment operators

The equal sign is used to assign a value to a variable. An assignment operator is a statement that assigns the right side value to the identifier that is on the left of the “=” sign, and the most basic form of this statement is *Variable_name=Expressions*. Other assignment operators are listed in Table 1.9.

Table 1.9 Assignment operators

Operator	Description
=	Assigns values from right-side operand to left-side operand.
+=	It adds right operand to the left operand and assigns the result to left operand.
-=	It subtracts right operand to the left operand and assigns the result to left operand.
*=	It multiplies right operand with the left operand and assigns the result to left operand.
/=	It divides left operand with the right operand and assigns the result to left operand.
%=	It takes modulus using two operands and assigns the result to left operand.
**=	Performs exponential (power) calculation on operators and assigns value to left operand.
//=	It performs floor division on operators and assigns value to the left operand.

Example 19

```
a = 10
print("The value of a is ",a)
a += 10
print("The value of a after += is ",a)
a -= 10
print("The value of a after -= is ",a)
a *= 10
print("The value of a after *= is ",a)
a /= 2
print("The value of a after /= is ",a)
a **= 2
print("The value of a after **= is ",a)
a //= 10
print("The value of a after //= is ",a)
```

Output

```
The value of a is 10
The value of a after += is 20
The value of a after -= is 10
The value of a after *= is 100
The value of a after /= is 50.0
The value of a after **= is 2500.0
The value of a after //= is 250.0
```

1.6.3 Comparison (relational) operators

Comparison operators compare the values on both sides and determine the relationship between them. They are also called relational operators. If the condition satisfies, it returns True, otherwise returns False (Table 1.10).

Table 1.10 Relational operators

<i>Operator</i>	<i>Description</i>
<code>=</code>	If the values of two operands are equal, then the condition becomes true.
<code>!=</code>	If values of two operands are not equal, then the condition becomes true.
<code><></code>	If values of two operands are not equal, then the condition becomes true.
<code>></code>	If the value of the left operand is greater than the value of the right operand, then the condition becomes true.
<code><</code>	If the value of the left operand is less than the value of the right operand, then the condition becomes true.
<code>>=</code>	If the value of the left operand is greater than or equal to the value of the right operand, then the condition becomes true.
<code><=</code>	If the value of the left operand is less than or equal to the value of the right operand, then the condition becomes true.

Example 20

```

a = 10
b = 2
print("The value of a and b is ",a,b)
print(a," == ", b ,a == b)
print(a," != ", b, a != b)
print(a," > ", b, a > b)
print(a," < ", b, a < b)
print(a," >= ", b, a >= b)
print(a," <= ", b, a <= b)

```

Output

```

The value of a and b is 10 2
10 == 2 False
10 != 2 True
10 > 2 True
10 < 2 False
10 >= 2 True
10 <= 2 False

```

1.6.4 Logical operators

The meaning of these operators is the same as we use them in the English language. Table 1.11 lists the Python logical operators and as it can be seen, there are three operators in this set. They are *and*, *or* and *not*.

Table 1.11 Logical operators

Operator	Description
and	True if both operands are true.
or	True if either of the operands is true.
not	Complements the operand.

Example 21

```

a=20
b=10
c=30
if a >= b and a >= c:
    print("a is big")
elif b >= a and b >= c:
    print("b is big")
else:
    print("c is big")

```

Output

```

c is big

```

1.6.5 Membership operators

There are two membership operators to choose from. They are *in* and *not in*. The *in* operator returns *True* if an item is part of a series, whereas the *not in* operator returns *False* if the item is not a part of a series.

Example 22

```
lst = [1,2,3,4,5]
no1 =3
no2 = 10
print(no1," is in ",lst,no1 in lst)
print(no2," is not in ",lst,no2 not in lst)
```

Output

```
3 is in [1, 2, 3, 4, 5] True
10 is not in [1, 2, 3, 4, 5] True
```

1.6.6 Identity operators

We examine if two variables or objects are identical, that is, if they point to the same memory address. The operators *is* and *is not* are called identity operators.

Example 23

```
x = 5
print(type(x) is int)
print(type(x) is not int)
```

Output

```
True
False
```

1.6.7 Bitwise operators

There are six bitwise operators: bitwise AND, bitwise OR, bitwise XOR, bitwise NOT(Complement), bitwise SHIFT TO THE LEFT and bitwise SHIFT TO THE RIGHT (Table 1.12).

Table 1.12 Bitwise operators

Operator	Description
&	Bitwise AND
	Bitwise OR
^	Bitwise XOR
~	Bitwise NOT (complement)
<<	Bitwise left shift
>>	Bitwise right shift

Example 24

```

x = 50
y = 2
print("x & y = ", x & y)
print("x | y = ", x | y)
print("x ^ y = ", x ^ y)
print("~x = ", ~x)
print("x << y = ", x << y)
print("x >> y = ", x >> y)

```

Output

```

x & y = 2
x | y = 50
x ^ y = 48
~x = -51
x << y = 200
x >> y = 12

```

1.6.8 Python operators precedence

The precedence of all the operators is followed as per Table 1.13.

Table 1.13 Precedence of operators

Operator	Meaning
()	Parentheses
**	Exponentiation
~,+, -	Complement, unary plus, unary minus
*,/, %, //	Multiply, divide, modulo, whole (floor) division
+, -	Addition, subtraction
<<, >>	Right, left bitwise shift
&	Bitwise and
^,	Bitwise exclusive OR, bitwise OR
<=, <, >, >=	Relational operators
<>, ==, !=	Equality operators
=, %=, /=, //=, -=, +=, *=, **=	Assignment operators
is, is not	Identity operators
in, not in	Membership operators

1.7 FLOW CONTROL STATEMENTS

The control flow of statements is divided into two categories: branching and looping. Conditional statements are another name for branching statements. They change the order in which program statements are executed. The statements that branch are:

1. if-else statement
2. if-elif-else statement

Looping structures, also known as iterating statements, are portions of code that are repeated until a termination condition is met. The looping statements are as follows:

1. for loop
2. while loop

If a condition is met when branching or looping, a certain block of instructions will be performed. In Python, indentation is used to express these blocks. The rule of indentation for expressing a block of code is known as off-side notation for coding. A tabbed piece of code is used as a delimiter, followed by a colon.

1.7.1 Conditional statements

Decision-making entails anticipating situations that may arise during program execution and describing actions to be done in response to such conditions. The conditional structures analyze several expressions and return *True* or *False* as the result. Based on the result of the conditional expression (True or False), we must decide which action to take or which statements to execute.

This is the syntax for an if statement in Python:

```
if Conditional_Expression :  
    Statement(s)1  
else:  
    Statement(s)2
```

In the if-else statement notice that the keyword *if* is used to begin the if-else sentence. The conditional expression returns one of two values: True or False. A colon (:) is necessary. Following the colon, begins the body of the *if*, which consists of one or more statements. All of these statements must also be tabbed. Any sentence that is not tabbed and follows the body of the if-else clause is not included. If the conditional expression returns *True*, the statements in the if body are performed. If the conditional expression returns False, the lines in the else body are performed. The else clause in the if-else statement is optional (Figure 1.1).

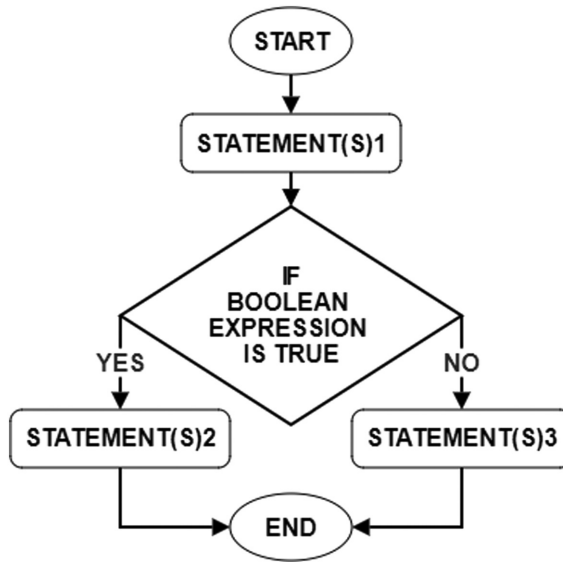


Figure 1.1 Flowchart for if condition

Example 25

```

a=10
b=15
if a < b:
    print (a, "is small")
else:
    print (b, "is small")
  
```

Output:

```
10 is small
```

1.7.2 The elif statement

When we need to verify several conditions, we write if–elif–else statements. It's worth noting that “else if” is spelled “elif” in Python (Figure 1.2).

This is the syntax for the if–elif–else statement in Python is:

```

if Conditional_Expression1 :
    Statement(s)1
elif Conditional_Expression2 :
    Statement(s)2
else:
    Statement(s)3
  
```

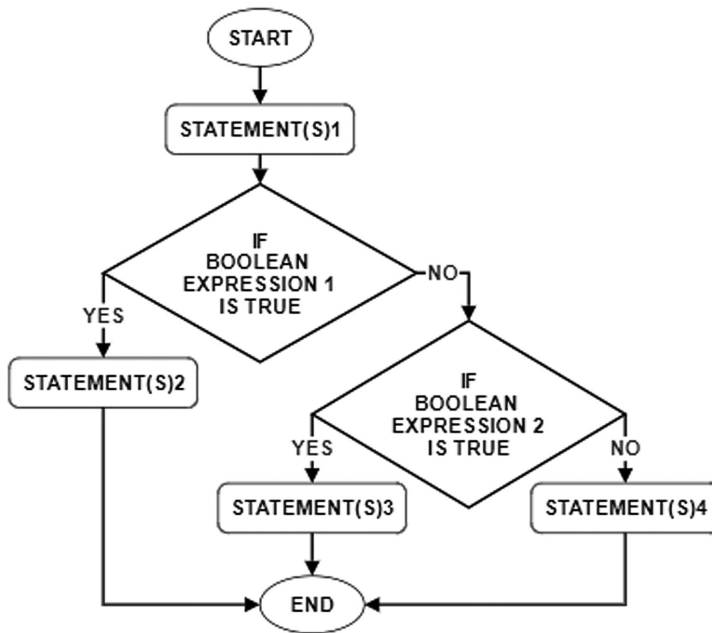


Figure 1.2 Flowchart for elif condition

Example 26

```
my_list1 = ["Hi", "How", "are", "You"]
my_list2 = ["I", "am", "Fine"]
my_input = input("Enter your String ")
if (my_input in my_list1):
    print(my_input, "is in L1st1")
elif (my_input in my_list2):
    print(my_input, 'is in List2')
else:
    print(my_input, 'is NOT available in any List')
```

Output 1

```
Enter your String Hi
Hi is in L1st1
```

Output 2

```
Enter your String am
am is in List2
```

Output 3

```
Enter your String Happy
Happy is NOT available in any List
```

1.7.3 Ternary operator

The ternary operator is a simple if–else statement with only one line of code. The ternary operator is used instead of multiline if–else expressions.

The syntax for a simple ‘ternary operator’ statement is:

Expression1 if Conditional_Expression else Expression2

Example 27

```
a = int(input("Enter First Number "))
b = int(input("Enter Second Number "))
print ("The bigger number is:", a) if a > b else print("The
bigger number is:",b)
```

Output

```
Enter First Number 10
Enter Second Number 5
The bigger number is: 10
```

1.7.4 Looping statement

Iterated (repeated) execution of statements is referred to as looping. In Python, there are two types of looping structures:

1. while loop
2. for loop

1.7.4.1 While loop

While the conditional expression is true, the while loop repeats a series of statements. Because the limit for terminating the loop is unknown in advance, this looping structure is referred to as indefinite.

The syntax for while loop is:

while conditional_Expression :

Statement(s)1

else:

Statement(s)2

In the while loop, notice that, the while loop is initiated with the keyword *while*. The conditional expression returns True or False. The conditional expression does not have to be enclosed by a pair of parentheses. A colon (:) is usually necessary. Following the colon, the following line begins the body of the while loop, which consists of one or more statements. All of these statements must also be tagged. Any statement that is not tabbed and

follows the body of the while loop is not a part of it. When the conditional expression evaluates to True, the statement(s)1 in the while-loop body is performed. Also, be aware that if the while conditional loop's expression constantly evaluates to true (never becomes false), the loop will never finish. A loop that never ends is referred to as an endless loop. The while-else loop's clause is optional. When the control variable's value is not in the collection of items, the statement(s)2 inside the else block is executed (Figure 1.3).

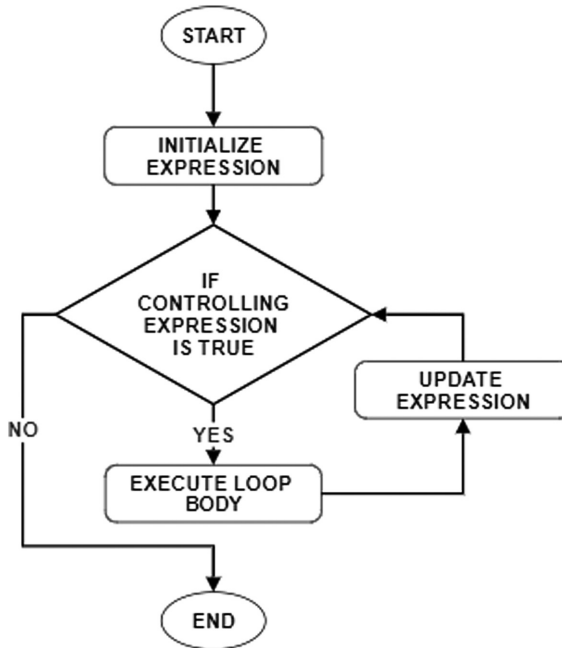


Figure 1.3 Flowchart for while loop

The while-loop is used when we need to repeatedly execute some statements until some conditions are met. We don't know in advance when this condition will be met.

Example 28

```

i = 0
while i < 5 :
    print(i, "I \m Learning Python")
    i = i + 1
else:
    print("inside the while-else")
    print(i, "I \m Enjoying it.")
  
```

Output

```

0 I 'm Learning Python
1 I 'm Learning Python
2 I 'm Learning Python
3 I 'm Learning Python
4 I 'm Learning Python
inside the while-else
5 I 'm Enjoying it.

```

Example 29

```

my_num = int(input("Enter a Number : "))
fact = 1
i = 1
while (i <= my_num):
    fact = fact * i
    i = i + 1
print("The factorial of ", my_num, " is ", fact)

```

Output

```

Enter a Number : 7
The factorial of 7 is 5040

```

1.7.4.2 For loop

Another method for iterating (repeating) statement execution is the for loop. The for loop is used in two situations: first, when we want to run over each item in a sequence and, second, when we want to repeat an action a specific number of times. This looping form is characterized as a definite iteration since there is a defined boundary for loop termination.

The syntax of the for loop is:

for Variable_Name in Collection_of_Items :

Statement(s)1

else:

Statement(s)2

It is important to remember that the for loop starts with the keyword *for*. The loop's control variable (or iterating variable) is any variable name following the for keyword; it counts the loop's turns automatically. The *in* keyword allows you to apply values to the control variable using the element mentioned in the collection of items. The range() function, which is responsible for producing all of the essential control variable values, may be used to get the collection of items variable. This method creates integer sequences from only numbers as input. The usage of a colon is always required. Following the colon, the body of the for loop, which consists of one or more statements. All of these claims must be tagged as well.

Statements that are not tabbed and follow the body of the for loop are not included in the for loop. The else condition in the for loop is optional. When the value of the control variable is not found in the collection of items, the statement(s) within the else block is performed.

When we know the maximum number of times the body of the loop will be run, we utilize the for loop (Figure 1.4).

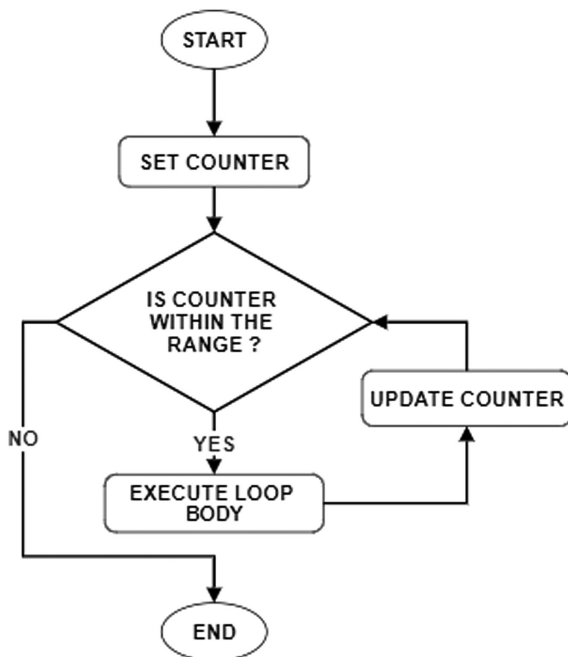


Figure 1.4 Flowchart for for loop

1.7.4.3 The range() function

The range function accepts three arguments, two of which are optional. The first argument is Start, which is the first number in the range; if this parameter is not specified, the default value is 0. The second parameter is End, which is the range's final value; the end value cannot be omitted. If the Increment/Decrement option is missing, the increment is set to 1.

Range function	Range object (output)
<code>range(10)</code>	0, 1, 2, 3, 4, 5, 6, 7, 8, 9
<code>range(1, 10)</code>	1, 2, 3, 4, 5, 6, 7, 8, 9
<code>range(1, 10, 2)</code>	1, 3, 5, 7, 9
<code>range(10, 0, -1)</code>	10, 9, 8, 7, 6, 5, 4, 3, 2, 1
<code>range(10, 0, -2)</code>	10, 8, 6, 4, 2

<i>Range function</i>	<i>Range object (output)</i>
<code>range(2, 11, 2)</code>	2, 4, 6, 8, 10
<code>range(-5, 5)</code>	-5, -4, -3, -2, -1, 0, 1, 2, 3, 4
<code>range(1, 2)</code>	1
<code>range(1, 1)</code>	(empty)
<code>range(1, -1)</code>	(empty)
<code>range(1, -1, -1)</code>	1, 0
<code>range(0)</code>	(empty)

Example 30

```
for i in range(5):
    print(i, "I \'m Learning Python")
else:
    print("inside the for-else")
    print(i, "I \'m Enjoying it.")
```

Output

```
0 I 'm Learning Python
1 I 'm Learning Python
2 I 'm Learning Python
3 I 'm Learning Python
4 I 'm Learning Python
inside the for-else
4 I 'm Enjoying it.
```

Example 31

```
my_num = int(input("Enter a Number : "))
fact = 1
for i in range (1,my_num+1):
    fact = fact * i
print("The factorial of ", my_num," is ", fact)
```

Output

```
Enter a Number : 7
The factorial of 7 is 5040
```

1.7.4.4 Continue, break and pass

The *continue* statement skips the rest of the code within the loop for the current iteration alone. The loop does not end, but instead continues with the following iteration. When the keyword *continue* appears, the control merely continues to execute for the next value of the control variable without doing anything. *Continue* is used in both the for loop and the while loop.

Example 32

```
for i in range(5):
    if(i == 3):
        continue
    print(i)
print("Out of Loop")
```

Output:

```
0
1
2
4
Out of Loop
```

The *break* statement ends the loop that contains it. When a *break* statement is encountered within a loop, control moves to the sentence immediately after the body of the loop.

Example 33

```
for i in range(5):
    if(i == 3):
        break
    print(i)
    i += 1
print("Out of Loop")
```

Output:

```
0
1
2
Out of Loop
```

The term *pass* refers to doing nothing (no code). It serves as a stand-in. It means that instead of nothing, we write *pass*.

Example 34

```
for i in range(5):
    if(i == 3):
        pass
    else:
        print(i)
print("Out of Loop")
```

Output:

```
0
1
2
4
Out of Loop
```

1.7.4.5 Nested loops

A nested loop is one that is contained within another loop. Any loop can be nested inside of another loop. This indicates that a *for* loop can be nested within another *for* loop, or a *for* loop can be nested inside a *while* loop, or

a while loop can be nested inside another while loop, or a while loop can be nested inside a for loop.

Example 35

```
lower = int(input("Enter lower range: "))
upper = int(input("Enter upper range: "))
if(lower < upper):
    print("Prime numbers between ",lower," and ",upper," are :")
    for num in range(lower, upper):
        p = 1
        for i in range(2, num):
            if(num % i) == 0:
                p = 0
                break
        if(p == 1):
            print(num, "is a Prime Number")
```

Output

```
Enter lower range: 8
Enter upper range: 25
Prime numbers between 8 and 25 are :
11 is a Prime Number
13 is a Prime Number
17 is a Prime Number
19 is a Prime Number
23 is a Prime Number
```

1.7.6 Iterator

The iterator (iter) is a distinct data structure in Python. It may “iterate” through the sequence, beginning at 0 and ending at the last item. Python supports iterating not only over sequences, but also over non-sequence data types (keys in a dictionary, lines in a file, and so on), as well as user-defined objects. An iterator has a method named next that is unique to it (). As with every looping structure, we increment the control variable to get to the next value. The iterator__next__() method is used to obtain the next item in an iterator. After all of the objects have been exhausted, the iterator throws a Stop exception.

Example 36

```
MyTup=(1, 'two', 3.0, 4j,"Five")
i = iter(MyTup)
print("The next item in Tuple is ",i.__next__())
```

Output

```
The next item in Tuple is 1
```

1.8 FUNCTIONS

There are various situations when we need to repeat a job several times while building software. Consider a program that computes a number's factorial at various times throughout the program. It would be inefficient and time-consuming to have to enter the same code for calculating the factorial every time we required it. The solution is to give each piece of code a name and then refer to it whenever it is required.

1.8.1 Defining a function

The general syntax for a function is:

```
def Function_Name([parameter list]) :  
    body_of_the_function  
    [return]
```

Take note of the following in a function: It is usually preceded by the term *def* (for “define”). Following *def* is the function's name (the rules for naming functions are the same as for naming variables). A pair of parentheses () is put next to the function name. Input parameters, which are optional, are specified inside the parentheses. It is possible to have a function with no input arguments if it is optional. If more than one parameter is specified, they are separated by a comma. The line must be terminated with a colon, which is always necessary. The function's body is made up of one or more statements. All of the statements within the function's body are tabbed. Any statement that is not tabbed will be excluded from the function. Finally, there is a return statement that is optional.

1.8.2 Calling a function

A function cannot work by itself. It only works when we give it a command. The next step is to call the function by name. When invoking the function, we should provide the necessary values in the parentheses.

Example 37

```
def sum(a,b):  
    c = a + b  
    print("The sum is ",c)  
  
A = int(input("Enter First number : "))  
B = int(input("Enter Second number : "))  
sum(A,B)
```

Output

```
Enter First number : 12
Enter Second number : 5
The sum is 17
```

1.8.3 Returning results from a function

By providing a “return” statement in the function body, we may get the function’s result or output. We don’t need to include a return statement in the body of a function if it returns nothing.

Example 38

```
def sum(a,b):
    c = a + b
    return c

A = int(input("Enter First number : "))
B = int(input("Enter Second number : "))
print("The sum is ",sum(A,B))
```

Output

```
Enter First number : 12
Enter Second number : 5
The sum is 17
```

1.8.4 Returning multiple values from a function

A function in Python can return several values. When a function computes several outcomes and wants to return the results, we may use the return statement as *return a, b*.

Here two values, *a* and *b*, are returned. The function returns these values as a tuple. To obtain these values, we may use two variables when invoking the function as: *x,y= functionName ()*.

Example 39

```
def Arith_Op(a,b):
    c = a + b
    d = a - b
    e = a * b
    f = a / b
    return c,d,e,f

A = int(input("Enter First number : "))
B = int(input("Enter Second number : "))
sum, sub, mul, div = Arith_Op(A,B)
print("The sum is ", sum)
print("The subtraction is ", sub)
print("The multiplication is ", mul)
print("The division is ", div)
```

Output

```
Enter First number : 15
Enter Second number : 3
The sum is 18
The subtraction is 12
The multiplication is 45
The division is 5.0
```

1.8.5 Pass by value

The phrase *pass by value* means that the function is given a copy of the variable value, and any changes to that value will not be reflected outside the function. In Python, object references are used to provide values to functions. Everything is considered an object in Python. All numbers, strings, tuples, lists and dictionaries are considered objects.

Example 40

```
def change(a):
    a = a + 5
    print("The value of a inside the function ",a)

a = int(input("Enter First number : "))
print("The value of a before calling the function ",a)
change(a)
print("The value of a after calling the function ",a)
```

Output:

```
Enter First number : 15
The value of a before calling the function 15
The value of a inside the function 20
The value of a after calling the function 15
```

1.8.6 Pass by reference

The phrase *pass by reference* refers to passing the variable's reference or memory address to the function. Because the function updates the variable value through memory address, the new value is reflected both inside and outside the function.

Example 41

```
def change(a,b):
    a.append(b)
    print("The list inside the function ",a)

lst = [5,10,15,20]
print("The list before calling the function ",lst)
change(lst,25)
print("The list after calling the function ",lst)
```

Output:

```
The list before calling the function [5, 10, 15, 20]
The list inside the function [5, 10, 15, 20, 25]
The list after calling the function [5, 10, 15, 20, 25]
```

1.8.7 Formal and actual arguments

A function may have some parameters. These parameters are useful to receive values from outside of the function. They are called “formal arguments.” When we call the function, we should pass data or values to the function. These values are called “actual arguments.”

In the previous example, *change(lst,25)* has *lst* and *25* as actual arguments, whereas *change(a,b)* has *a* and *b* as formal arguments. Sometimes actual arguments are simply known as arguments, whereas formal arguments are known as parameters.

The actual arguments are of four types: positional, keyword, default and variable.

1. Positional arguments

Assigning the *i*th argument to the *i*th parameter is called the positional parameter; such arguments are called positional arguments.

Example 42

```
def Function_with_Parameter(a,b):
    c = a + b
    print("The sum of",a,"and",b,"is",c)
Function_with_Parameter(5,10)
```

Output

The sum of 5 and 10 is 15

Here the position of the values of the argument is important. When we called the function for the first time the arguments were 5 and 10. These values are passed as parameters. Inside the function parameter *a* takes the value 5, and parameter *b* takes the value 10.

2. Keyword arguments

Python has another pattern for passing arguments, termed keyword argument passing, in which the meaning of the argument is determined by its name.

Example 43

```
def Function_with_Parameter(a,b):
    c = a + b
    print("The sum of",a,"and",b,"is",c)
Function_with_Parameter(a=5,b=10)
```

Output:

The sum of 5 and 10 is 15

Remember that the values supplied to the parameters are preceded by the names of the target parameters, followed by the equal sign (=) and lastly a value. It makes no difference in which order we supply the arguments. We are unable to utilize a name for a parameter that does not exist. If we don't, we'll get an error message. The value of each argument knows where it is going because of the name. In both situations, the outcome is the same.

3. Default arguments

Instead of sending value as an argument, we can assign a value to the parameter. Assume that the value of some argument is always the same. In that case, we can provide the argument a value. The rest of the code can be provided as an argument (may be positional or may be keyword argument). The term *default parameter* refers to such parameters.

Example 44

```
def Function_with_Parameter(a,b,c=40):  
    sum1 = a + b + c  
    print("The sum of",a,"",b,"and",c,"is",sum1)  
  
Function_with_Parameter(10,20)  
Function_with_Parameter(10,b=30)
```

Output:

```
The sum of 10 , 20 and 40 is 70  
The sum of 10 , 30 and 40 is 80
```

The function contains three arguments, as you can see. One of the parameters has the value 40 (i.e., $c = 40$). The remaining two parameters must be given to the function when it is called (may be positional or may be keyword argument).

4. Variable length arguments

The programmer might not be aware of how many values a function can receive at any given time. The programmer is unable to identify the number of arguments to provide in the function definition. A variable length parameter is utilized in the function definition for this reason. A variable length argument is one that may be filled with any number of values. In the function definition, the variable length parameter is written with a and asterisk (*) before it:

```
def add(farg, *args)
```

Example 45

```
def sum_of_numbers(*no):
    sum1 = 0
    for i in no:
        sum1 += i
    return sum1
print("The sum is", sum_of_numbers())
print("The sum is", sum_of_numbers(4))
print("The sum is", sum_of_numbers(34,2))
print("The sum is", sum_of_numbers(15,50,52))
```

Output

```
The sum is 0
The sum is 4
The sum is 36
The sum is 117
```

1.8.8 Recursive functions

Recursion is a problem-solving technique. It involves breaking down a problem into smaller and smaller sub-problems until we get to a small enough problem that can be solved quickly. A technique in which a function calls itself is known as recursion.

There are three recursion laws:

1. A recursive function must provide a base case that indicates when the function terminates.
2. A recursive function must progress toward the base case.
3. A recursive function must also call itself.

Example 46

```
def rec_fun_factorial(a):
    if a<1:
        return None
    if a == 1:
        return 1
    else:
        return rec_fun_factorial(a-1)*a
x = int(input("Enter a number: "))
print("The factorial of", x,"is", rec_fun_factorial(x))
```

Output:

```
Enter a number: 10
The factorial of 10 is 3628800
```

Observe that, the terminating condition is “if the value of ‘a’ is less than 1, it returns None and if the value of a is equal to 1” the factorial is 1. Otherwise we call the function recursively to find the factorial.

1.8.9 Anonymous function or lambdas

A lambda function is one that has no name. The name “lambda” or “anonymous” comes from the fact that they are not stated as other functions using the *def* keyword. Rather, the *lambda* keyword is used to generate them. Lambda functions are temporary functions that only need to be utilized where they were generated. They may be used everywhere a function is needed. Because of the desire from LISP programmers, the lambda functionality was added to Python.

The general syntax of the lambda function is:

lambda parameters : expression

It should be noted in a lambda function that the term *lambda* is used to start a lambda function. The arguments are placed next to the keyword, which is optional. After it, there is a colon. After the colon, the phrase is written that needs to be evaluated. The lambda function is a function that returns a value.

Example 47

```
First= lambda a,b : a + b
no1 = int(input("Enter First Number : "))
no2 = int(input("Enter Second Number : "))
print("The sum is ", First(no1,no2))
```

Output:

```
Enter First Number : 12
Enter Second Number : 4
The sum is 16
```

With lambda functions, code becomes shorter, cleaner and more legible. The lambda function returns the value of the expression implicitly, so we don't need to add a return statement directly. We can send a function as a parameter to another function on occasion. Lambda functions are the best option in these situations.

We can use lambda functions very commonly with inbuilt functions like *filter()*, *map()* and *reduce()*.

1.8.10 Function decorators

A decorator is a function that takes a parameter and returns another function. A decorator takes a function's result, modifies it and then returns it. As a result, decorators come in handy when a function requires some extra processing. In most cases, the following stages are involved in the development of decorators:

- A decorator function with another function name as an argument should be defined.

- Within the decorator function, we should define a function. The value of the function provided to the decorator function is modified or decorated by this function.
- The inner function that processed or decorated the value is returned.

The current functions are given new meaning by the decorator. In other words, decorators wrap one function around another. Decorators can also be used on functions that are specified by the user.

Decorator is a function that takes a function as an argument and extends the functionality of that function, returning a modified function with the additional functionality. The behavior of the original function remains unchanged while the functionality is extended.

Example 48

```
def improved_Div(func):
    def inner(a,b):
        if a < b and a==0:
            return func(0,b)
        elif b == 0:
            return func(0,a)
        elif a < b:
            return func(b,a)
        else:
            return func(a,b)
    return inner

@improved_Div
def Div(a,b):
    return a/b

n,m= eval(input("Enter two numbers(separated by comma):"))
print("The division is", Div(n,m))
```

Output 1

```
Enter two numbers(separated by comma):15,6
The division is 2.5
```

Output 2

```
Enter two numbers(separated by comma):0,6
The division is 0.0
```

Output 3

```
Enter two numbers(separated by comma):23,0
The division is 0.0
```

1.8.11 Function generators

A generator is a function that is responsible for generating a set of values. We may build generator functions in the same way we write regular functions, but the *yield* keyword is used to return values. Iterators are used to get one value at a time from a database.

The yield statement can be used to hold and return a succession of outcomes.

Example 49

```
def even(n):
    i = 1
    while (i <= n):
        if (i%2==0):
            yield i
        i += 1
j = even(10)
for k in j:
    print(k)
```

Output

```
2
4
6
8
10
```

When working with huge amounts of data, it is often preferable to utilize a generator rather than a function. This is due to the fact that a function returns a huge amount of data at once, all of which must be put into memory (sometimes the memory may not be sufficient). And it's possible that we'll only be dealing with one dataset at a time. In this case, we may utilize a generator that only produces one data at a time. Instead of collecting the full dataset, we may fetch one data at a time using a generator.

1.8.12 Modules

We also have the option of creating our own module. It's comparable to developing a standard program when we create our own modules. Two files are required. The first one is the module itself. The file name is the module name and must have a .py extension. The second file contains the code for the new module.

Let us create a module "My_Module.py". It has two functions *MY_add(a,b)* and *My_mul(a,b)*.

Example 50

```
def MY_add(a,b):
    return a + b

def My_mul(a,b):
    return a * b
```

We can call the functions defined in *my_module.py* by importing the module.

Example 51

```
import my_module

a=5
b=10
x= my_module.MY_add(a,b)
print("The sum is",x)
y= my_module.My_mul(a,b)
print("The multiplication is", y)
```

1.9 CLASSES AND OBJECTS

A class defines data as well as functions. This is accomplished by creating a class that consists of two sections: a declaration part and an implementation part. The declaration part specifies variables and functions, whereas the implementation part defines functions whose prototypes were specified in the declaration section. This is known as encapsulation, and it means that the data and the functions are linked together. We know that a class is a template of an object.

The general syntax for a class is:

```
class ClassName:
    'class documentation string'
    class_suite
```

Notice that the term *class* is used to begin the definition. The keyword *class* is followed by an identifier, which is the class's name, and then a colon (:). Inside the class, the class suite is described. Everything in the class suite folder should be tabbed appropriately. The properties and behavior are both contained in the class suite.

The variables are called attributes, and the methods specified in the class are called behaviors. Each class is split into three sections:

Within its own namespace, a class has a name that uniquely identifies it. Each class has its own set of attributes.

A class consists of a collection of operations. It is the capacity to carry out certain tasks.

It is possible to leave the *class_suite* empty, a class with no attributes and no methods.

Example 52

```
class Two_Wheeler:
    Make= 'Hero'

    def __init__(self):
        self.Regd_no= 'PY 37 PQ 7898'
        self.owner= 'Srusti'
        self.kind= 'Scooter'

    def details(self):
        print ('Owner ',self.owner, 'has a',self.kind,'with Regd
no',self.Regd_no)
```

In this example, the *class_suit* contains attributes (variables) and methods (behaviors). Here *Make*, *Regd_no* and *owner* are attributes, whereas *__init__()* and *type()* are methods.

A class definition is nothing more than a design or template for all of the objects. Having a class is therefore pointless if no object is created. In other terms, an object is a class instance.

The presence of a class does not imply that all suitable objects will be generated automatically. We must build the object because the class cannot do it.

The general syntax for creating an object is: *Object_name=Class_name()*

The object provided is an instance of the class we requested. When we use the functional notation to assign a class, the newly generated object inherits all of the class's properties. Instantiation is the process of generating an object of the chosen class (as the object becomes an instance of the class).

Example 53

```
My_Byke = Two_Wheeler ()
My_Byke.details()
Two_Wheeler.details(My_Byke)
```

Output

```
Owner is : Srusti has a Scooter with no PY 37 PQ 7898
Owner is : Srusti has a Scooter with no PY 37 PQ 7898
```

There are two different ways we can use the methods defined in a class.

Method 1: Keep in mind that *My_Byke* is the object generated for the class *Two_Wheeler* in the first line. The second line, *My_Byke.details()*, invokes a method provided within the class. Remember that *My_Byke* is an object of *Two_Wheeler*, and the *details()* method of *My_Byke* accepts the parameter "self". It implies we wish to use a method that is part of the *My_Byke* object.

Method 2: The *details()* is a method defined in the *Two_Wheeler* class. We must give the object name as a parameter to access the methods specified within the class. There might be a lot of objects produced from the same class. The object name must be supplied as an argument to identify which

object is calling the method. The method *details()* is accessible by object *My Byke*, which is provided as an argument to access the method defined in the class employee, in the third line, *Two_Wheeler.details(My_Byke)*.

1.9.1 `__init__()` , `__new__()` and `__del__()`

There are two methods, `__init__()` and `__new__()`, used to initialize certain data and constructor of an object, whereas `__del__()` is used to destruct an object.

1.9.1.1 `__init__()`

This method's primary goal is the initialization of an object.

The general syntax is:

```
__init__(self [,arguments])
```

It's worth noting that the method's name is "`__init__()`" and it's preceded and followed by two underscores (`__`). The method's parameter is *self* and in addition to it a list of arguments may be supplied. It does not return any results.

The argument *self* is not passed to the method when it is called. Python does it automatically. The argument *self* refers to the object itself. The object that has invoked the method is this one. This indicates that the function `__init__()` accepts no parameters and should accept *self*. In the same way, a method that accepts just one argument will really accept two: *self* and the argument.

When an object is created, the first method that is automatically run is this one. The `__init__()` function is invoked every time an object is created. This function may be used to initialize variables in a class object.

There are two ways of assigning values to the variable in an object.

Method 1: Inside the `__init__()` method, we can assign the value to the variable statically.

Example 54

```
class Two_Wheeler:
    def __init__(self):
        self.Regd_no= 'PY 37 PQ 7898'
        self.owner = 'Srusti'
        self.kind = 'Scooter'

My_byke = Two_Wheeler()
print(My_byke.Regd_no,My_byke.owner,My_byke.kind)
```

Take note of the method `__init__(self)`. The value 'PY 37 PQ 7898' is set to `self.Regd_no`, 'Srusti' is set to `self.owner` and 'Scooter' is set to `self.kind` in

this case. We did not explicitly use the `__init__()` method while instantiating the object `My_byke`. Automatically, variables are assigned to the values supplied inside the method. The *self* refers to the object that is calling the method, which in this instance is `My_byke`.

Method 2: A value can be passed to the `__init__()` method that can assign the value to the variable dynamically.

Example 54

```
class Two_Wheeler:
    def __init__(self, regd, owner, kind):
        self.Regd_no = regd
        self.owner = owner
        self.kind = kind

My_byke = Two_Wheeler('PY 37 PQ 7898', 'Srusti', 'Scooter')
print(My_byke.Regd_no, My_byke.owner, My_byke.kind)
```

Observe that the `__init__(self, regd, owner, kind)` method has three more parameters in addition to the parameter *self*. We supplied the values for the `self.Regd_no`, `self.owner` and `self.kind` while creating the object `My_byke = Two_Wheeler('PY 37 PQ 7898', 'Srusti', 'Scooter')`.

The output is the same in both cases. The value of `Regd_no`, `owner` and `kind` is fixed for all the objects created in the first case, but the second method allows for more flexibility by supplying the value as an argument.

1.9.1.2 `__new__()`

The main purpose of this method is object creation. The general syntax is:

*object.__new__(cls[, *args, **kwargs])*

or

*super(class_name, cls).__new__(cls[, *args, **kwargs])*

It is worth noting that the method's name is "`__new__()`" and it's preceded and followed by two underscores (`__`). In the method, the argument is *cls*. A list of arguments can be supplied in addition to *cls*. It returns an object.

It is a static method that returns a new object instance that accepts *cls* as the first parameter. This approach is referred to as a "constructor." When the new class is ready to be instantiated, this method is invoked. This technique is used to modify an object's creation.

Example 55

```

class Two_Wheeler(object):
    def __new__(cls, regd, owner, kind):
        if regd.find('TS') != -1 or regd.find('ts') != -1:
            return object.__new__(cls)
        else:
            return None
    def __init__(self, regd, owner, kind):
        self.Regd_no= regd
        self.owner= owner
        self.kind = kind

    def display(self):
        print('Registration Number :',self.Regd_no, 'Owner :',self
            .owner, 'Kind :',self.kind )

My_byke= Two_Wheeler('TS 04 DE 1234','Aruna Kranthi','Byke')
My_byke.display()

```

Output

```
Registration Number : TS 04 DE 1234 Owner : Aruna Kranthi Kind : Byke
```

cls, regd, owner and kind are the arguments in the `__new__()` function. The method `__new__()` is a static method, as indicated by the cls argument. When the value of regd is *TS* or *ts*, the object is created. When the object is created, it returns the object together with its data, and when the object was not generated, it returns *None*.

Before the `__init__()` function is invoked, the `__new__()` method is called. It's because value initialization is only possible when an instantiation happens. When `__new__()` returns a class instance, the `__init__()` method is executed automatically. This instance is given as a parameter to the `__init__()` function. When utilizing the `__new__()` method, the `__init__()` function is not necessary to initialize some value to a variable.

Example 56

```

class Two_Wheeler(object):
    def __new__(cls, regd, owner, kind):
        if regd.find('TS') != -1 or regd.find('ts') != -1:
            self= object.__new__(cls)
            self.Regd_no = regd
            self.owner= owner
            self.kind = kind
            return self
        else:
            return None

    def __str__(self):
        return '{0}{1}'.format(self.__class__.__name__,
            self.__dict__)

My_byke= Two_Wheeler('TS 04 DE 1234','Nakshatra','Byke')
print(My_byke.Regd_no,My_byke.owner,My_byke.kind)
My_bykel= Two_Wheeler('OD 23 EE 1034','Aruna Kranthi','Scooter')
print(My_bykel)

```

Output

```
TS 04 DE 1234 Nakshatra Byke
None
```

In the `__new__()` method, notice how we generated objects based on the value of `Regd_no`. When an object is successfully created, the value given as a parameter is assigned to `self.regd_no`, `self.owner`, and `self.kind`. Then `self` is returned.

The variables of the object `My_byke` may be displayed using `print(My_byke.Regd_no, My_byke.owner, My_byke.kind)`. Because `My_byke1` isn't defined, it returns `None`, and attempts to display the value using `My_byke1.Regd_no` and `My_byke1.owner` will result in an `AttributeError`. When using the `__new__()` method to create objects, overloading the `__str__()` function is usually recommended.

1.9.1.3 __del__()

The goal of the destructor is to clean up the object before the garbage collector destroys it.

The general syntax is:

```
object.__del__(self):

# the destructor's body
```

When all references to an object are removed, this function is invoked. Garbage collection refers to the process of deleting unreferenced items from memory. When an object's reference count is 0, the Python interpreter runs the garbage collector to free it from memory.

Example 57

```
class Test:
    def __init__(self,a):
        self.no = a
        print("Object ",a, " is Created")

    def __del__(self):
        print("Object is Deleted")

obj1 = Test(1)
obj1 = 5

obj2 = Test(2)
del obj2

print("Program End")
```

Output

```
Object 1 is Created
Object is Deleted
Object 2 is Created
Object is Deleted
Program End
```

To destroy an object, we must either explicitly invoke the *del* method or erase the reference.

Observe how *obj1 = Test(1)* destroys the object it produced. As soon as *obj1* refers to 5, the destructor function is executed automatically. When *del obj2* is run, the object produced by *obj2 = Test(2)* is also deleted.

If more than one reference pointing to the same object, then the destructor cannot be called unless all the references are removed.

Example 58

```
class Test:
    def __init__(self,a):
        self.no = a
        print("Object ",a, " is Created")

    def __del__(self):
        print("Object is Deleted")

obj1 = Test(1)
obj2 = obj1
del obj1
print("Program End")
```

Output

```
Object 1 is Created
Program End
```

Observe that *obj1* and *obj2* both refer to the same object, which was created by *obj1 = Test (1)*. Because the object is still referenced by *obj2*, the command *del obj1* did not call the destructor.

1.10 VARIABLES AND METHODS

The variables and the methods are an important part of any class.

1.10.1 Variables (attributes)

Object variables and class variables are the two types of variables. The class variables are held by the class, but the object variables are owned by each object.

1.10.1.1 Object variables (instance variables)

Variables that vary from object to object are known as instance variables. Each object makes a duplicate of the variable. There are three ways to generate an instance variable.

1. Inside Initializer by using self-variable.
2. Outside the class, using the object reference variable.
3. Inside Instance Method by using self-variable.

Example 59

```
class Two_Wheeler:
    def __init__(self):
        self.Regd_no= 'PY 37 PQ 7898'

    def kin(self):
        self.kind = 'Scooter'

My_byke= Two_Wheeler()
My_byke.owner = 'Nakshatra'
My_byke.kin()
print("Owner : ", My_byke.owner, " has a ", My_byke.kind," with
    redg no : ",My_byke.Regd_no)
```

Output

```
Owner : Nakshatra has a Scooter with redg no : PY 37 PQ 7898
```

In this example, we declare instance variables inside the initializer `__init__()` by using `self`. Once we create an object, automatically these variables will be added to the object. The statement `self.Regd_no= 'PY 37 PQ 7898'` defines a variable and assigns a value to it.

The value to the variables (attributes) can also be assigned after the object is created. It is done by referencing the object. The general syntax is ***Object_name.variable_name = value***. The statement `My_byke.owner = 'Nakshatra'` defines one more variable and assigns a value to it.

If any instance variable is declared inside the instance method, that instance variable will be added to the object once we call that method. Observe the method `kin()`, inside that method `self.kind = 'Scooter'` statement defines a variable `self.kind`. In order to access the variable, we need to call the method.

1.10.1.2 Class variables (static variables)

If the value of a variable does not change from object to object, it must be defined within the class but outside of any method. Variables of this sort are known as static variables or class variables. These variables are linked to the class object to which they belong and are not affected by any class

instances. Only one copy of a static variable is produced for a class, and it is shared by all objects of that class.

We can create a static variable in five different ways:

1. Inside the class.
2. Inside the constructor by using class name.
3. Inside instance methods by using class name.
4. Inside static methods by using class name.
5. Inside class methods by using either class name or *cls* variable.

Example 60

```
class Two_Wheeler:
    make = 'Hero'
    def __init__(self):
        self.Regd_no= 'PY 37 PQ 7898'
        Two_Wheeler.kind = 'Scooter'

    def colr(self):
        Two_Wheeler.color = 'Red'

    def capacity():
        Two_Wheeler.engine = '100 CC'

    @classmethod
    def seat(cls):
        cls.seater = 2

My_byke= Two_Wheeler()
My_byke.owner = 'Nakshatra'
My_byke.colr()
Two_Wheeler.capacity()
My_byke.seat()
print("Owner : ", My_byke.owner, " with redg no : ",My_byke.
    Regd_no)
print("Make :",My_byke.make,"Colour :", Two_Wheeler.col
    or,"Seater :",Two_Wheeler.seater)
```

Output:

```
Owner : Nakshatra with redg no : PY 37 PQ 7898
Make : Hero Colour : Red Seater : 2
```

In this example all the five ways of class variables are defined. The statement *make = 'Hero'* is inside the class definition but not inside any method that defines a class variable. Inside the *__init__()* method, the statement *Two_Wheeler.kind = 'Scooter'* defines a class variable.

Inside *colr(self)*, which is an instance method, the statement *Two_Wheeler.color = 'Red'* defines a class variable.

Inside *capacity()*, which is a static method, the statement *Two_Wheeler.engine = '100 CC'* defines a class variable. Inside *seat(cls)*, which is a static method, the statement *cls.seater = 2* defines a class variable.

1.10.1.3 Accessing variables

It is not enough to just declare variables and assign values to them; we must also be able to access them. In the software, there are several ways to access these variables.

The different methods for accessing variables are:

1. Accessing the variable by object reference.
2. Accessing the variable by class name.
3. Accessing the static variable outside the class by class name.

Example 61

```
class Two_Wheeler:
    make = 'Hero'
    def __init__(self):
        self.Regd_no= 'PY 37 PQ 7898'
        Two_Wheeler.kind = 'Scooter'

    def colr(self):
        Two_Wheeler.color = 'Red'

    @classmethod
    def seat(cls):
        cls.seater = 2

My_byke= Two_Wheeler()
My_byke.owner = 'Nakshatra'
My_byke.colr()
My_byke.seat()
print("Owner : ", My_byke.owner, " with redg no : ",My_byke.
Regd_no)
print("Make :",My_byke.make,"Colour :", Two_Wheeler.col
or,"Seater :",Two_Wheeler.seater)

My_byke1= Two_Wheeler()
My_byke1.Regd_no= 'TS 12 RR 3256'
My_byke1.owner = 'Srusti'
Two_Wheeler.make = 'Enfield'
Two_Wheeler.color = 'Black'
Two_Wheeler.seater = 1
print("Owner : ", My_byke.owner, " with redg no : ",My_byke1.
Regd_no)
print("Make :",My_byke1.make,"Colour :", Two_Wheeler.col
or,"Seater :",Two_Wheeler.seater)
```

Output

```
Owner : Nakshatra with redg no : PY 37 PQ 7898
Make : Hero Colour : Red Seater : 2
Owner : Nakshatra with redg no : TS 12 RR 3256
Make : Enfield Colour : Black Seater : 1
```

We can access the variable by object reference. The general syntax to access a variable is *object_name.variable_name*. Observe that we have defined a

class variable *make*. In order to access the variable we can use the object reference that is `My_byke.make` and `My_byke1.make`.

When accessing a static variable, we can use the class name instead of the object name. Because the static variable is shared by all the objects generated, we may access it using the class name. To access a static variable, use the general syntax *class_name.variable_name*. Observe that we have access to the same static variable by writing `Two_Wheeler.color` and `Two_Wheeler.seater`.

Static variables can also be accessed outside of the class. We may access a class variable in the same manner we assigned values to it. *class_name.variable_name* is the standard syntax for accessing a static variable outside the class. Observe that there is a class variables *make*, *colour* and *seater* defined inside the class changed to 'Enfield' , 'Black' and 1, respectively after the creation of the object. There may be several class variables defined for a class; we need to mention for which class variable the value is changed. It is also possible to define a class variable the same way we changed the value of a class variable.

1.11 PUBLIC, PRIVATE AND PROTECTED VARIABLES

Variables can be classified as public, private or protected. These are also called *access specifiers*.

Public variables are variables specified in a class that can be accessed using the dot operator from anywhere in the program. Private variables, on the other hand, are variables that have a single or double underscore prefix in the class definition. The strongly private variables are only accessible from within the class and not from anywhere else.

Unlike most programming languages, Python does not contain private, public or protected keywords; instead, they are distinguished by the absence of an underscore, a single underscore or a double underscore symbol before the variable or method.

If there is no underscore before a variable's or a method's name, it means the name is public. As a result, it is open to everyone and may be accessed from anywhere in the program.

If a variable or method has an underscore before its name, it means it is exclusively for internal usage and may be changed whenever the class wishes.

Python does not truly protect these names. As a result, these names can be accessed directly from other modules. Such variables are sometimes referred to as “weak private” or “protected.” These names can be used within the class, the child class and the package, but not outside of the package.

If a variable or function has a double underscore (__) before its name, it is for internal use only and may be changed only within the class. Such names are referred to as “strong private.”

Because we cannot access private members directly from another class, we can use such variable names many times in various methods. The main purpose of the double underscore (__) is to utilize names (variable/method) exclusively within the class if they are not to be used outside of the class.

Example 62

```
class Two_Wheeler:
```

```
    def __init__(self, regd, knd, mk):
        self.Regd_no= regd
        self._kind = knd
        self.__make = mk

    def show(self):
        print("From show() method:")
        print(self.Regd_no)
        print(self._kind)
        print(self.__make)
```

```
My_byke= Two_Wheeler('PY 37 PQ 7898', 'Scooter', 'Hero')
My_byke.show()
print("Data saved by My_byke Object \n", My_byke.__dict__)
print("\nFrom Outside the class")
print(My_byke.Regd_no)
print(My_byke._kind)
print(My_byke.__make)
```

Output

```
From show() method:
PY 37 PQ 7898
Scooter
Hero
```

```
Data saved by My_byke Object
{'Regd_no': 'PY 37 PQ 7898', '_kind': 'Scooter', '_Two__make': 'Hero'}
```

```
From Outside the class
PY 37 PQ 7898
Scooter
```

```
-----
AttributeError                                Traceback (most recent call last)
<ipython-input-38-d00836b25c11> in <module>()
    19 print(My_byke.Regd_no)
    20 print(My_byke._kind)
--> 21 print(My_byke.__make)
    22
```

```
AttributeError: 'Two_Wheeler' object has no attribute '__make'
```

Observe that the variable *Regd_no* has no underscore, the variable *kind* (*_kind*) has a single underscore and the variable *make* (*__make*) has two underscores. We can see that these variables can be accessed from within the class (*show()* method), and from outside the class the variables *My_byke.Regd_no* and *My_byke._kind* can be accessed. Whereas, accessing *My_byke.__make* outside the class raised *AttributeError*. The *My_byke.__dict__* shows the internal storage of data in the object *My_byke*.

1.12 METHODS

Under the object oriented paradigm, a function is known as a method. A method is a function that belongs to a certain class. Methods are specified inside the context of a class. The relationship between the class and the method is made explicit in this method declaration. The syntax for calling a function differs from that for calling a method.

There are three different types of methods:

1. Instance method
2. Class method
3. Static method

1.12.1 Instance method

The term *instance method* refers to a method that works with the instance variable. As an argument to the instance method, *self* must be provided. Other than *self*, we may or may not have any other arguments.

In the parameter list, the first argument is always *self*. Furthermore, if the method only has one argument, *self*, we do not supply any value for this parameter when calling the method. Similarly, a method that accepts just one argument will really accept two parameters, *self* and the *parameter*.

Example 63

```
class Two_Wheeler:
    def __init__(self, regd, knd, mk):
        self.Regd_no= regd
        self.kind = knd
        self.make = mk

    def show(self):
        print("From show() method:")
        print("Registration no : ",self.Regd_no)
        print("Kind of Two Wheeler : ",self.kind)
        print("Make of the Two-Wheeler : ",self.make)

My_byke= Two_Wheeler('PY 37 PQ 7898', 'Scooter', 'Hero')
My_byke.show()
```

Output

```
From show() method:
Registration no : PY 37 PQ 7898
Kind of Two Wheeler : Scooter
Make of the Two-Wheeler : Hero
```

In this, *self* is a part of the parameter, so `__init__()` and `show()` are instance methods.

1.12.2 Class method

We'll need a *class method* to deal with the class variable. In two ways, class methods vary from regular methods. They are first called by a class (not by the instance of the class). Second, the class method's first parameter is *cls* rather than *self*. In most cases, class methods are used to instantiate a class with arguments other than those given to the class constructor.

Example 64

```
class Two_Wheeler:
    make = 'Hero'

    def __init__(self, regd, knd):
        self.Regd_no= regd
        self.kind = knd

    def mk_value(cls):
        return cls.make

    def show(self):
        print("From show() method:")
        print("Registration no : ",self.Regd_no)
        print("Kind of Two Wheeler : ",self.kind)
        print("Two wheeler maker : ",Two_Wheeler.
mk_value(Two_Wheeler))

My_byke = Two_Wheeler('PY 37 PQ 7898', 'Scooter')
My_byke.show()
```

Output

```
From show() method:
Registration no : PY 37 PQ 7898
Kind of Two Wheeler : Scooter
Two wheeler maker : Hero
```

It's worth noting that the class variable *make* is specified. A class method is required to access this class variable. Because it accepts *cls* as an argument, the method `mk_value(cls)` is a class method. We can see that we have typed `Two_Wheeler.mk_value(Two_Wheeler)` to retrieve the returned value of this class method. It indicates that there is a class function `mk_value(cls)` in the employee class, and we must give the class name *Two_Wheeler* as a parameter.

1.12.3 Static method

Static methods are used when we require a method that has nothing to do with the instance or class variables. This method can be used to interact with a different class or object.

A *static method* is any functionality that belongs to a class but is not required by the objects. The main distinction between static and class methods is that static methods do not take any extra parameters. They perform the same functions as regular class methods.

Example 65

```
class Two_Wheeler:
    make = 'Hero'

    def __init__(self, regd, knd):
        self.Regd_no= regd
        self.kind = knd

    def mk_value(cls):
        return cls.make

    def show(self):
        print("From show() method:")
        print("Registration no : ",self.Regd_no)
        print("Kind of Two Wheeler : ",self.kind)
        print("Two wheeler maker : ",Two_Wheeler.
mk_value(Two_Wheeler))

    @staticmethod
    def print_info():
        print("\nFrom Static method")
        print("This is printed for static method")

My_byke = Two_Wheeler('PY 37 PQ 7898','Scooter')
My_byke.show()
Two_Wheeler.print_info()
```

Output

```
From show() method:
Registration no : PY 37 PQ 7898
Kind of Two Wheeler : Scooter
Two wheeler maker : Hero

From Static method
This is printed for static method
```

We can make a method static by using a decorator *@staticmethod* and the static method is accessed by *class_name.method_name()*. Observe that we have a method *print_info()*; this method is a static method, as we have used the decorator *@staticmethod*.

1.13 CLASS INSIDE A CLASS (INNER CLASS)

Defining a class inside another class is sometimes required. The term *inner class* describes such a class. It is self-evident that we require an outside class in order to define an inner. For instance, departments are found at every college. There is no department class unless there is a college class. As a result, the department class must be part of the college's inner circle.

We can create the object of the inner class in two ways:

1. The object of the inner class is created inside the outer class.
2. The object of the inner class is created outside the outer class.

Example 66

```
class Two_Wheeler:
    make = 'Hero'

    def __init__(self, regd, own):
        self.Regd_no= regd
        self.owner = own
        self.details = self.byke()

    class byke:
        def __init__(self):
            self.make = 'Hero'
            self.engine = '100 CC'
            self.kind = 'Scooter'

    class colr:
        def __init__(self):
            self.color = 'Red'

My_byke = Two_Wheeler('TS 77 AA 4325', 'Tusarika')
clr = Two_Wheeler.colr()
print("Owner : ", My_byke.owner, "Registration no : ", My_byke.
      Regd_no )
print("Make of the Two Wheeler : ", My_byke.details.make)
print("Two wheeler maker : ", My_byke.details.engine)
print("Two wheeler maker : ", My_byke.details.kind)
print("Color of the two wheeler : ", clr.color)
```

Output

```
Owner : Tusarika Registration no : TS 77 AA 4325
Make of the Two Wheeler : Hero
Two wheeler maker : 100 CC
Two wheeler maker : Scooter
Color of the two wheeler : Red
```

For the outer class *Two_Wheeler*, we have defined two inner classes: *byke* and *colr*. The object of inner class is created inside the outer class by *self*. *InnerClassObjectName = self.InnerClassName()*. The implementation is done by *self.details = self.byke()*.

The object of the inner class can also be created outside the outer class by *InnerClassObjectName = OuterClassName.InnerClassName()*. The implementation is done by *clr = Two_Wheeler.colr()*.

1.13.1 Accessing attributes of inner class

Accessing a name defined inside the inner method is a bit tricky. The different ways of accessing the attributes of inner class are:

- i. Accessing the attributes of inner class outside the class.
- ii. Accessing the attributes of inner class in the outer class.
- iii. Accessing the attributes of inner class outside the outer class.
- iv. Accessing the attributes of inner class when inner class and outer class have the same method.

Example 67

```
class Two_Wheeler:
    make = 'Hero'

    def __init__(self, regd, own):
        self.Regd_no= regd
        self.owner = own
        self.details = self.byke()

    def show(self):
        print("\nAttributes of inner class inside outer class:")
        print("Registration no : ",self.Regd_no)
        print("Owner of Two Wheeler : ",self.owner)
        print("Two wheeler maker : ",self.details.make)
        print("\nAttributes of inner class when inner and outer
            class have same method:")
        self.details.show()

class byke:
    def __init__(self):
        self.make = 'Hero'
        self.engine = '100 CC'
        self.kind = 'Scooter'

    def show(self):
        print("\nFrom show() method inside inner class:")
        print("Registration no : ",self.regd)
        print("Owner of Two Wheeler : ",self.own)
        print("Two wheeler maker : ",self.make)

class colr:
    def __init__(self):
        self.color = 'Red'

My_byke = Two_Wheeler('TS 77 AA 4325','Tusarika')
clr = Two_Wheeler.colr()
My_byke.show()

print("\nAttribute of Inner class From Outside the outer
    class:")
print("Two wheeler engine : ",My_byke.details.engine)
print("Two wheeler maker : ",My_byke.details.kind)
print("\nAttribute of Inner class Outside the outer class:")
print("Color of the two wheeler : ",clr.color)
```

Output

Attributes of inner class inside outer class:

```
Registration no : TS 77 AA 4325
Owner of Two Wheeler : Tusarika
Two wheeler maker : Hero
```

Attributes of inner class when inner and outer class have same method:

From show() method inside inner class:

```
Registration no : TS 77 AA 4325
Owner of Two Wheeler : Tusarika
Two wheeler maker : Hero
```

Attribute of Inner class From Outside the outer class:

```
Two wheeler engine : 100 CC
Two wheeler maker : Scooter
```

Attribute of Inner class Outside the outer class:

```
Color of the two wheeler : Red
```

Here the outer class is *Two_Wheeler*, and there are two inner classes *byke* and *clr*. The outer class *Two_Wheeler* has attributes 'Regd_no' and 'owner'. The inner class *byke* has attributes 'make', 'engine' and 'kind'. The inner class *clr* has attribute 'color'.

In order to access the attribute of the inner class outside the class, the general syntax is

ObjectofOuterClass.ObjectofInnerClass.AttributeNameofInnerObject.

This is implemented in *My_byke.details.engine* and *My_byke.details.kind*.

In order to access the attribute of the inner class outside the outer class, the general syntax is

self.ObjectofInnerClass.AttributeNameofInnerObject.

This is implemented in *self.details.make*.

We have already seen that an object of inner class can be created outside the outer class. In order to access the attribute of the inner class outside the outer class, the general syntax is

ObjectofInnerClass.AttributeNameofInnerObject.

This is implemented in *clr.color*.

If a method has the same name in both inner and outer class, we can call the inner class *self.innerClassObjectName.innerClassMethod()*. This is implemented in *self.details.show()*.

True–false questions

1. Curly braces are used to identify a block of code in Python.
 - a. True
 - b. False

2. `a,b,c = 1,2,3` is a valid statement.
 - a. True
 - b. False
3. Python variables do not need explicit declaration to reserve memory space.
 - a. True
 - b. False
4. Decision-making is anticipation of conditions occurring while execution of the program does not specify actions taken according to the conditions.
 - a. True
 - b. False
5. An `else` statement must come with an `if` statement.
 - a. True
 - b. False

Fill-in-the-blank questions

1. A _____ is a function that accepts a function as a parameter and returns a function.
2. _____ arguments are passed to a function in correct positional order.
3. An _____ is an object that stores a group of elements of the same datatype.
4. An expression is a combination of _____, _____ and _____ written according to the syntax of Python language.
5. _____ in Python are identified as a contiguous set of characters represented in the quotation marks. Python allows for either pairs of single or double quotes.

Multiple-choice questions

1. The value of the expression `16%7%4` will be evaluated to
 - a. 1
 - b. 2
 - c. 3
 - d. 0
2. Find the output of the following code

```
print(15 << 2)
```

 - a. `15 * 2`
 - b. `15 * (2+2)`
 - c. `15 * (2**2)`
 - d. `15*(2+3)`
3. Find the output of the following code:

```
x = 4
x == 3 + 2
print(x)
```

- a. 5
 - b. 4
 - c. True
 - d. False
4. The output of `print(float('100.0'))` is
- a. 100.0
 - b. 100
 - c. Error
 - d. None of the above
5. The right-side binding means the expression will be evaluated:
- a. From left to right
 - b. From right to left
 - c. In random order
 - d. None of the above
6. Find how many times “+” will be printed in the output of the following code:
- ```
i = 0
while i <= 5:
 i+=1
 if i % 2 == 0:
 break
 print("+")
```
- a. 0
  - b. 1
  - c. 2
  - d. 3
7. Find the output of the following code:
- ```
x = 10
if x > 5:
    x = x + 5
if x < 12:
    x = x + 5
if x == 15:
    x = x + 5
print(x)
```
- a. 10
 - b. 15
 - c. 20
 - d. 25

8. Find the output of the following code:

```
x = 10
y = 0
z = y < x and x > y or y > x and x < y
print(z)
a. True
b. False
c. TRUE
d. FALSE
```

9. Find the output of the following code:

```
my_list = ["pet", "dog", 35, "cat", 23]
count = 0
for item in my_list:
    if type(item) == str:
        continue
    count = count + 1
print(count)
a. 1
b. 3
c. 4
d. 2
```

10. Find the output of the following code:

```
for i in range(1):
    print("**")
else:
    print("$")
a. One * only
b. Two * only
c. One * and a $
d. Two * and a $
```

11. Find the output of the following code:

```
sum = 0
x = [i**2 for i in range(3)]
l = len(x)
for i in range(l):
    sum += x[i]
print(sum)
a. 2
b. 3
c. 4
d. 5
```

12. Find the output of the following code:

```
s = 'Python is easy'
s1 = s[-7:-5]
s2 = s[-4:-2]
print(s1 + s2)
a. isea
b. is easy
c. easyeasy
d. iseasyeasy
```

13. Find the output of the following code:

```
s = 'Python is Fun'
s1 = s[6:-4]
print(len(s1))
a. 2
b. 3
c. 4
d. 5
```

14. Find the output of the following code:

```
i = [10,20,[30,40],[50,60]]
count = 0
for i in range(len(l)):
    if type(l[i]) == list:
        count = count + 1
print(count)
a. 1
b. 2
c. 3
d. 4
```

15. Find the output of the following code:

```
a = ()
print(bool(a))
a. False
b. True
c. Depends on processor
d. Not defined
```

16. Find the output of the code:

```
d1 = {1:'one',2:'two'}
d1.pop('two')
a. ValueError
b. AttributeError
```

- c. KeyError
 - d. None of the above
17. Find the output of the following code:
- ```
set1 = {1,2,3,4}
set1.add(2)
print(set1)
```
- a. {1,2,3,4}
  - b. {1,2,2,3,4}
  - c. All of the above
  - d. None of the above
18. Function is defined inside
- a. Another function
  - b. Module
  - c. Class
  - d. All of the above
19. Consider the following code:
- ```
def get_names():
    names = ['Parrot','Owl','Peacock','Maina','Crow']
    return names[2:]
def update_names(elements):
    new_names = []
    for name in elements:
        new_names.append(name[:3].upper())
    return new_names
print(update_names(get_names()))
```
- Find the output.
- a. ['PEA']
 - b. ['PAR','PEA']
 - c. ['PAR','PEA','OWL']
 - d. ['PAR','PEA','OWL','MAI']
20. Consider the following code:
- ```
def calculate(amount = 6, factor = 3):
 if amount > 6 :
 return amount * factor
 else:
 return amount * factor *2
```
- Which among the following statements will give output 30?
- a. calculate()
  - b. calculate(10)
  - c. calculate(5,6)
  - d. calculate(6,3)

**Descriptive questions**

1. Write a Python program that calculates compound interest.
2. Write a Python program that swaps two numbers using a third variable.
3. Write a program that finds the greatest common divisor (GCD), or highest common factor (HCF), of two positive numbers.
4. Write a program to remove duplicate values from the dictionary.
5. Write a Python program that will add square brackets to a list. If the list contains a single item ['Peacock'] and we want three more square brackets [ ], the output should be [ [ [ 'Peacock' ] ] ] .
6. Write a Python program to remove an empty tuple(s) from a list of tuples.

**Answers to true–false questions**

1. False
2. True
3. True
4. False
5. False

**Answers to fill-in-the-blank questions**

1. decorator
2. Positional
3. array
4. variables, constants, operators
5. strings

**Answers to multiple-choice questions**

---

|     |   |     |   |     |   |     |   |     |   |
|-----|---|-----|---|-----|---|-----|---|-----|---|
| 1.  | b | 2.  | c | 3.  | b | 4.  | a | 5.  | b |
| 6.  | b | 7.  | c | 8.  | a | 9.  | d | 10. | c |
| 11. | d | 12. | a | 13. | b | 14. | b | 15. | a |
| 16. | c | 17. | a | 18. | d | 19. | a | 20. | b |

---